

January 2026 CSE 406: Cyber Security Sessional

Online 2: Ethical Hacking

Section: C1

Time: 40 minutes

Ethical-use warning

This is an intentionally vulnerable, isolated teaching environment. Use the techniques in this assignment only against the supplied local containers. Never test an application, account, or network without explicit authorization. The same actions against a real service may be unethical and illegal.

Objective

You have access to a **Course Registration** portal and a **Study Group Chat**. Acting only through the designated attacker account, complete this cascading attack:

Extract every student's outstanding registration balance through SQL injection. Then persist a stored-XSS payload in your own Course Registration record. When you next check your real registration status, that payload must use the active Chat session to submit a second script as a chat message. The second script must remain stored in the Chat and execute automatically for a different user who later loads the chat feed, changing the page title to **C1-SECOND-ORDER-XSS**. It must only create this visible marker; it must not create further messages or replicate itself.

Environment Setup

The Moodle archive contains `C1-images.tar`, `compose.yaml`, and this specification. From the directory containing those files, run:

```
seed@ubuntu:~$  
  
docker load --input C1-images.tar  
docker compose up -d  
docker compose ps
```

Wait until all three services are running. Open:

- Course Registration: <http://localhost:3000>
- Study Group Chat: <http://localhost:3001>

To reset all database changes, sessions, and chat messages:

```
seed@ubuntu:~$  
  
docker compose down  
docker compose up -d
```

The 40-minute limit includes image loading and environment startup. Raise your hand immediately if Docker reports an infrastructure error before either page opens.

Credentials

Role	Student ID	Registration PIN	Chat user	Chat password
Attacker	9900001	reg-isha-29	isha.sen	chat-Isha-29
Victim	9900002	reg-fahim-29	fahim.mirza	chat-Fahim-29

Tasks and Marks

1. **Database extraction (2 marks):** Demonstrate SQL injection and extract every student's ID, name, and outstanding balance through the attacker submission channel.
2. **First stored XSS (2 marks):** Use a stacked statement to persist the first browser payload in student 9900001's own registration record.
3. **CSRF carrying executable content (3 marks):** A normal status check by the attacker must cause the Chat to store a second script message under the attacker's active Chat session. Manually posting the script does not count.
4. **Independent-viewer execution (3 marks):** Log in to the Chat as the victim, without using the victim on Course Registration, and demonstrate that simply loading the chat feed changes the title to C1-SECOND-ORDER-XSS. The stored script must be non-replicating.
5. **Bonus - cover your tracks (+1):** Discover the forensic trail left by registration submissions and delete only records attributable to student 9900001 through the same injection channel. All other evidence must remain.

Restrictions

- All SQL injection, extraction, persistent modification, and cleanup used for grading must be submitted with Student ID **9900001**.
- The victim account exists only to prove that the second stored payload reaches an independent Chat viewer. The victim must not access Course Registration during the proof.
- Direct database edits or manually posting the second script do not satisfy the required chain.
- Do not attack anything outside the supplied local environment or interfere with another student's workstation.

Submission

Create a plain-text file named with your seven-digit student ID, for example `2105XXX.txt`. Include every exact payload, its application and input field, the complete extracted roster, and an ordered explanation of how the first stored payload transports the second stored payload into the Chat. State how you verified execution under the independent victim session. Submit only this text file through Moodle.